



Taller de Programación Literaria en Python con Emacs

Jhanavi Molina

2026-04-30

1 Objetivos

- Familiarizar al estudiante con el uso de Python dentro de Org mode en Emacs.
- Practicar programación literaria combinando texto, matemáticas y código ejecutable.
- Usar `C-c C-c` para ejecutar bloques y `C-c '` para editar bloques en un buffer especializado.
- Resolver ejercicios sencillos de Python inspirados en temas frecuentes de cheat sheets: variables, tipos, listas, condicionales, bucles y funciones.

2 Instrucciones Generales

1. Realice todas las actividades en Linux o WSL.
2. Abra este archivo en Emacs y guárdelo con un nombre como `Taller-Python-Literario-(Apellido 0 Grupo).org`.
3. Ejecute bloques de código con `C-c C-c` cuando el cursor esté dentro del bloque. Org mode ejecuta el bloque y coloca el resultado en una sección `#+RESULTS`.
4. Para editar cómodamente un bloque de Python use `C-c '`. Esto abre un buffer de edición con el modo principal apropiado para el lenguaje; al usar `C-c '` nuevamente regresa al documento Org, y con `C-x C-s` se actualiza el contenido del bloque en el archivo base.
5. En este taller, en lugar de modificar código ya terminado, usted debe **escribir la lógica de programa** dentro de los bloques propuestos.
6. Exporte el documento a PDF con `M-x org-latex-export-to-pdf` cuando haya terminado.
7. Suba al aula virtual el archivo `.org` y el `.pdf`.

3 Recursos Adicionales

Puede consultar los siguientes recursos en Internet y en el Repositorio Git de la clase

- Este trabajo
- Tutorial de Python y Java en Emacs
- Tutorial $\text{\LaTeX}$ Python e Emacs
- The only Python cheatsheet you will ever need
- Comprehensive Python Cheatsheet
- Capítulo 2 del Libro *Practical Computer Architecture with Python and ARM*[1]
- Working with Source Blocks in Emacs

4 Cómo trabajar con bloques de Python

4.1 Flujo recomendado

1. Ubique el cursor dentro de un bloque `#+begin_src python`.
2. Presione `C-c '` para abrir el bloque en una zona de edición más cómoda con soporte del modo de Python en Emacs.
3. Escriba la lógica del programa dentro de ese buffer.
4. Guarde con `C-x C-s` para actualizar el bloque en el archivo Org.
5. Regrese con `C-c '` al documento principal.
6. Ejecute el bloque con `C-c C-c` para observar el resultado insertado por Org mode.

4.2 ¿Qué debe resolver el estudiante?

En cada actividad encontrará uno o más bloques de Python con comentarios que indican el objetivo. Su tarea consiste en:

- Leer la explicación del tutorial.
- Escribir el código solicitado dentro del bloque.
- Ejecutarlo con `C-c C-c`.
- Verificar la salida.
- Explicar brevemente lo observado cuando se le pida.

No se espera copiar código de memoria. Se espera construir soluciones pequeñas y entender la relación entre texto, código y resultados.

5 Parte 1: Tutorial guiado de Python en Org mode

Esta primera parte es de familiarización. El objetivo es practicar conceptos básicos de Python y usar repetidamente `C-c '` y `C-c C-c`.

5.1 Variables y tipos

Python permite almacenar valores en variables de diferentes tipos como enteros, flotantes y cadenas. Estos son temas introductorios comunes en hojas de referencia para principiantes.

5.1.1 Actividad 1

Escriba un programa que:

- Declare una variable entera llamada `edad`.
- Declare una variable flotante llamada `promedio`.
- Declare una cadena llamada `nombre`.
- Imprima el valor y el tipo de cada variable.

```
# Escriba aquí su programa
# Debe usar variables: edad, promedio, nombre
edad = 19
promedio = 18.99
nombre= 'Jhanavi'
print("Su edad es: ", edad)
print("Su promedio es: ", promedio)
print("Su nombre es: ", nombre)

Su edad es: 19
Su promedio es: 18.99
Su nombre es: Jhanavi
```

Explique en 2 líneas qué diferencia observa entre `int`, `float` y `str`: La variable `int` que son los números enteros, se usa más cuando son datos simples y no tan grades, en cambio la variable `float` acepta hasta los decimales, y la variable `string` se usa más para texto

5.2 Operaciones y expresiones

Python permite realizar operaciones aritméticas como suma, producto, potencia y residuo.

5.2.1 Actividad 2

Escriba un programa que:

- Defina dos variables numéricas `a` y `b`.
- Imprima su suma, resta, multiplicación, división y residuo.

```
# Escriba aquí su programa
a = 5
b = 7
suma = a + b
print("SUMA:", suma)

resta = a - b
print("RESTA:", resta)

division = a / b
print("DIVISION:", division)

residuo = a % b
print("RESIDUO:", residuo)

SUMA: 12
RESTA: -2
DIVISION: 0.7142857142857143
RESIDUO: 5
```

Escriba una línea indicando para qué sirve el operador `%`: Sirve para sacar el residuo de una división de dos números

5.3 Listas

Las listas permiten agrupar varios valores y recorrerlos. Las hojas de referencia introductorias suelen mostrar creación, acceso, modificación y recorrido de listas.

5.3.1 Actividad 3

Escriba un programa que:

- Cree una lista llamada `numeros` con cinco enteros.
- Imprima la lista completa.
- Imprima el primer y el último elemento.
- Imprima la longitud de la lista.

```
# Escriba aquí su programa
numeros = [3,4,6,7,10]
print("Lista completa: ", numeros)
print("Primer elemento: ", numeros[0])
print("Último elemento: ", numeros[4])
print("Longitud de la lista: ", len(numeros))
```

```
Lista completa:  [3, 4, 6, 7, 10]
Primer elemento:  3
Último elemento:  10
Longitud de la lista:  5
```

Explique en 1 o 2 líneas qué hace `len()`: Lo que hace `len()` es agarrar el tamaño o el número de elementos de una lista.

5.4 Bucles y condicionales

Los bucles permiten repetir acciones y los condicionales permiten tomar decisiones según una condición lógica.

5.4.1 Actividad 4

Escriba un programa que:

- Recorra los números del 1 al 10.
- Imprima cuáles son pares y cuáles son impares.

```
# Escriba aquí su programa
numeros = [1,2,3,4,5,6,7,8,9,10]
for i in range(0,len(numeros)):
    if(numeros[i]%2 == 0):
        print("Numero par")
        print(numeros[i])
    else:
        print("Numero impar")
        print(numeros[i])
```

```
Numero impar
1
Numero par
2
Numero impar
3
```

```
Numero par
4
Numero impar
5
Numero par
6
Numero impar
7
Numero par
8
Numero impar
9
Numero par
10
```

Describa brevemente cómo usó `if` y el operador `%`: Utilice el `if` dentro de un bucle `for` para hacer una condición usando `%` para saber si el módulo es 0 el número es par, si es diferente, es impar.

5.5 Comprensión de listas

La comprensión de listas es una manera concisa de crear listas en Python.

5.5.1 Actividad 5

Escriba un programa que:

- Genere una lista con los números del 1 al 10.
- Genere otra lista con los cuadrados de esos números usando **list comprehension**.
- Imprima ambas listas.

```
# Escriba aquí su programa
numeros = [1,2,3,4,5,6,7,8,9,10]
cuadrados = [n**2 for n in numeros]
print("Sus cuadrados son: ", cuadrados)
```

Sus cuadrados son: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Explique en 2 líneas qué ventaja tiene una **list comprehension** frente a escribir varios `append`: La ventaja de una **list comprehension** es que puede escribir en una sola línea el código o operación que querramos utilizar en cambio usar muchos `append` hace que el código sea más largo y difícil de ver el error si es que nos llegamos a equivocar.

5.6 Funciones

Las funciones permiten encapsular lógica y reutilizar código; son una parte básica y frecuente en resúmenes de Python.

5.6.1 Actividad 6

Escriba un programa que:

- Defina una función `cuadrado(x)`.
- La función debe retornar el cuadrado de `x`.

- Luego pruebe la función con 3 valores e imprima los resultados.

```
# Escriba aquí su programa
def cuadrado(x):
    numeroCuadrado = x**2
    print("Su cuadrado es: ", numeroCuadrado)
    return numeroCuadrado
cuadrado(8)
cuadrado(10)
cuadrado(5)

Su cuadrado es:  64
Su cuadrado es:  100
Su cuadrado es:  25
```

Escriba en 2 líneas por qué las funciones ayudan a organizar programas: Nos ayudan a optimizar más la operación o programa que querramos hacer, ya que nos facilitan el cálculo y son más fáciles de entender para convertir una operación compleja en algo más simple y eficiente.

5.7 Texto, matemáticas y código

Una ventaja de la programación literaria es documentar una idea con texto y validarla con código ejecutable en el mismo archivo.[cite:4]

Considere la fórmula de la suma de los primeros n números naturales:

$$S(n) = \frac{n(n+1)}{2}$$

5.7.1 Actividad 7

Escriba un programa que:

- Calcule la suma de 1 hasta n usando un bucle.
- Calcule la misma suma usando la fórmula.
- Compare los resultados para $n = 10$.

```
# Escriba aquí su programa
n = [1,2,3,4,5,6,7,8,9,10]
suma = 0
for i in range (0, len(n)):
    suma = suma + n[i]
print("Su suma es: " , suma)

#Usando la formula
n = 10
formula = (n*(n+1))/2
print("La suma con el uso de la formula es:", formula)

Su suma es:  55
La suma con el uso de la formula es: 55.0
```

Explique en 3 líneas cómo este ejemplo muestra la idea de programación literaria: La programación literaria explica lo que hay detrás de cada código o bucle, busca una razón lógica mediante una fórmula, al igual que se puede ocupar un bucle, los dos métodos dan el mismo resultado pero se pueden ocupar en distintas formas para poder entender.

6 Parte 2: Resolución de problemas sencillos

En esta segunda parte ya no se explica el concepto paso a paso. Ahora debe crear los bloques de código resolviendo pequeños problemas. Se recomienda usar `C-c` para editar cada bloque y `C-c C-c` para probar la solución.

6.1 Problema 1

Escriba un programa que lea una lista fija de calificaciones [7, 8, 10, 9, 6], calcule el promedio y lo imprima.

```
# Escriba aquí su solución
calificaciones = [7,8,10,9,6]
suma = 0
promedio = 0
for i in range(0, len(calificaciones)):
    suma = suma + calificaciones[i]
promedio = suma / len(calificaciones)
print("El promedio es: ", promedio)
```

El promedio es: 8.0

6.2 Problema 2

Escriba un programa que cuente cuántas vocales hay en la cadena "arquitectura".

```
# Escriba aquí su solución
palabra = "arquitectura"
cont = 0
for letra in palabra:
    if letra in "aeiou":
        cont = cont + 1
print("Hay:", cont, " vocales")
```

Hay: 6 vocales

6.3 Problema 3

Escriba un programa que construya una lista con los números pares del 2 al 20.

```
# Escriba aquí su solución
numeros = []
for i in range (2,21):
    if i % 2 == 0:
        numeros = numeros + [i]
print(numeros)
```

[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

6.4 Problema 4

Escriba una función `maximo_de_tres(a, b, c)` que retorne el mayor de tres números y pruébela con un ejemplo.

```
# Escriba aquí su solución
def maximo_de_tres(a,b,c):
    if a > b and a > c:
        print("El numero mayor es:", a)
    elif b > a and b > c:
        print("El numero mayor es:", b)
    else:
        print("El numero mayor es:", c)
maximo_de_tres(6,9,2)
El numero mayor es: 9
```

6.5 Problema 5

Escriba un programa que imprima la tabla de multiplicar del 5, desde 5 x 1 hasta 5 x 10.

```
# Escriba aquí su solución
n = 5
for i in range(1,11):
    tablaMultiplicar = (n * i)
    print(tablaMultiplicar)

5
10
15
20
25
30
35
40
45
50
```

6.6 Problema 6

Escriba un programa que determine si la palabra "reconocer" es palíndroma.

```
# Escriba aquí su solución
def es_palindromo(palabra):
    palabra = palabra.lower()
    invertida = ""
    for letra in palabra:
        invertida = letra + invertida
    return palabra == invertida

if es_palindromo("reconocer"):
    print("Es palindromo")
else:
    print("No es palindromo")

Es palindromo
```

7 Equipo de Trabajo

Complete la información de los integrantes del grupo e indique el líder.

Nombre	email	Rol
Molina Jhanavi	jhanavi.molina@epn.edu.ec	colaborador

8 Verificación de Entregables [100%]

- ☒ Parte 1 completada.
- ☒ Parte 2 completada.
- ☒ Cuestionario de 10 preguntas respondido en el aula virtual.
- ☒ Uso de `C-c` ' para editar bloques practicado.
- ☒ Ejecución de bloques con `C-c` `C-c` verificada.
- ☒ Archivo PDF generado con `M-x org-latex-export-to-pdf`.
- ☒ Entrega de archivo `.org` y `.pdf`.